Near-linear deterministic communication protocol for exact degeneracy and k-cores

Sepehr Assadi, Prantar Ghosh, Bruno Loff, Parth Mittal, Sagnik Mukhopadhyay

doi:10.4230/LIPIcs.CCC.2024.7; CC BY 4.0

Editorial note. Native author TeX from fixed arXiv v1, distributed under CC BY 4.0. All proof-bearing communication protocols retained, including the warm-up as supplied by the authors. Source-local graph definitions and standard peeling facts are included. Front matter and unrelated lower bounds omitted. Counted result is thm:communication-upper only.

Allowed standard prerequisites (editorial).

- Classical peeling characterization of degeneracy. Matula and Beck 1983, author source fact:min-k-ord and fact:peelingmin-degen retained

6.1 k-Cores and Degeneracy in Graph Streams

For any undirected graph $G = (V, E)$ and integer $k \geqslant 1$, a *k-core* in G is a maximal set S of vertices such that the induced subgraph of G on S, denoted by $G[S]$, has a minimum degree of at least k. In other words, any vertex in S has at least k other neighbors in S.

k-Cores provide a natural notion of well-connectedness in massive graphs, and as such, computing k-cores (and more generally k-core decompositions; see, e.g., [LSY+22]) has been widely studied in databases [BGKV14, CZL+20, LZZ+19], social networks [DBS17, DBS18, KBVT15], machine learning [ADBV05, ELM18, GLM19], among others [LWSX18, GBGL20, SGJ+13].

As a result, in recent years, there has been a rapidly growing body of work on computing k-cores on massive graphs in parallel and streaming models of computation [SGJ+13, DBS17, DBS18, ELM18, GLM19, LSY+22]. In particular, [ELM18] presented a single-pass algorithm that for any $\varepsilon > 0$, computes a $(1-\varepsilon)$-approximation of every k-core in G (i.e., obtains a $(1-\varepsilon)$-approximate k-core decomposition) in $\widetilde{O}(n/\varepsilon^2)$ space (see also [SGJ+13] for an earlier streaming algorithm and [GLM19] for a closely related parallel algorithm).

The *degeneracy* of a graph $G = (V, E)$, denoted by $\kappa(G)$, is the largest integer $k \geqslant 0$ such that G contains a non-empty k-core. The simple greedy algorithm that at every step peels off the smallest degree vertex results in the so-called *degeneracy ordering* of G and $\kappa(G)$ is equal to the largest degree of a vertex removed in this peeling process [MB83]. Degeneracy is a standard measure of uniform sparsity and is closely related to other such notions like arboricity (which is within a factor 2 of degeneracy). Moreover, computing degeneracy is a subroutine for approximating various other problems such as arboricity [AYZ97], densest subgraph [Cha00], $(\kappa + 1)$ coloring [EH66].

The degeneracy problem, and closely related uniform-sparsity measures such as densest subgraph, have also been studied extensively in the graph streaming literature [BKV12, FT14, BHNT15, MTVV15, FT16, BCG20, AA20]. In particular, [FT14] provided an $O(\log n)$-pass semi-streaming algorithm that outputs a constant factor approximation to degeneracy, and [FT16] subsequently improved this to a single-pass $(1-\varepsilon)$-approximation in $\widetilde{O}(n/\varepsilon^2)$ space (see also [MTVV15] for densest subgraph and [BCG20, AA20] for degeneracy coloring).

In terms of lower bounds, [BCG20] prove that any *single-pass* streaming algorithm that computes the exact value of degeneracy or approximates it within an additive factor of λ requires $\Omega(n^2)$ space or $\Omega(n^2/\lambda^2)$ space respectively. Our *polynomial-pass* lower bounds for k-cores and degeneracy, now in a very strong sense, rule out the possibility of extending any prior semi-streaming algorithms computing near-optimal solutions to these problems, to compute *exactly* optimal solutions.

6.2 Degeneracy and k-Cores

The definition of a k-core and the degeneracy $\kappa(G)$ of a graph G can be found in Section 6.1. Here, we define some related terms that come up often in the paper. We also state some basic facts about them that we later use in our proofs.

Given an ordering σ of vertices, we say $v \prec_\sigma u$ if u appears (somewhere) after v in σ.

Definition 6.1 (Outdegree with respect to an ordering)**.** For a graph $G = (V, E)$. given an ordering σ of vertices in V, the outdegree of a vertex v with respect to σ, denoted by $\text{odeg}_\sigma(v)$, is the number of neighbors of v that appear after it in σ. Formally,

$$\text{odeg}_\sigma(v) = |\{u \in N(v) : v \prec_\sigma u\}|$$

We can also think of it as orienting all edges (x, y) from x to y if $x \prec_\sigma y$. Then $\text{odeg}_\sigma(v)$ is the outdegree of v in the resultant oriented graph.

Definition 6.2 (k-ordering)**.** Given a graph $G = (V, E)$, an ordering σ of vertices in V is called a k-ordering if $\forall v \in V : \text{odeg}_\sigma(v) \leqslant k$.

The following fact connects vertex orderings to degeneracy.

Fact 6.3 ([MB83])**.** *The minimum number k for which a graph G admits a k-ordering is $\kappa(G)$.*

The above fact implies that to prove that a graph has degeneracy at most k, it suffices to exhibit a k-ordering of the nodes. We now formally define *degeneracy ordering* of a graph.

Definition 6.4 (Degeneracy ordering)**.** Given a graph $G = (V, E)$, a $\kappa(G)$-ordering of V is called a degeneracy ordering of G.

A degeneracy ordering of a graph can be obtained by running the following "peeling algorithm" on it: starting with an empty sequence σ, recursively remove the minimum degree vertex from G and append it to σ until the graph becomes empty. Again, the maximum degree that a node has when it is peeled by this algorithm is exactly $\kappa(G)$. If not, then the peeling algorithm gives a k-ordering for some $k < \kappa(G)$ which violates Fact 6.3. Hence, we get the following fact.

Fact 6.5. *Suppose the peeling algorithm on a graph G removes its nodes in the order $v_1, \ldots, v_n$. Define $G_0 := G$ and $G_i := G \setminus \{v_1, \ldots, v_i\}$ for $i \in [n]$. Then,*

$$\kappa(G) = \max_{i \in [n]} \deg_{G_{i-1}}(v_i)$$

.

7 Communication Upper Bounds

In this section, we will see a communication upper bound for computing the degeneracy of a graph exactly. The setting is as follows: Alice and Bob are respectively given *disjoint* edge sets E_A and E_B on the set of vertices V (known to both Alice and Bob), and wish to compute the degeneracy of the graph $G = (V, E_A \cup E_B)$. The main theorem of this section is:

Theorem 7.1. *There exists a deterministic communication protocol that, given an n-vertex graph G, computes the degeneracy $\kappa(G)$ using $O(n \log^3 n)$ bits of communication. Moreover, the protocol outputs a $\kappa(G)$-ordering of the vertices of G, and a $\kappa(G)$-core in G.*

We will focus on a decision version of the degeneracy problem, which we define in the lemma below:

Lemma 7.2. *There exists a deterministic communication protocol that given an n-vertex graph G and an integer k, decides whether the degeneracy $\kappa(G) \leqslant k$ using $O(n \log^2 n)$ bits of communication. Moreover, if the protocol accepts, it outputs a k-ordering of the vertices of G, and if it rejects, it outputs a $(k+1)$-core in G.*

For brevity, we will call the decision version of degeneracy defined in the lemma above $\mathsf{DEGEN}_{n,k}$. Note that once we have a protocol for the decision problem, a protocol for the search variant follows via binary search.

Proof of Theorem 7.1. Alice and Bob can binary search for the minimum k such that G has degeneracy at most k (which is exactly $\kappa(G)$) using the protocol given by Lemma 7.2. Note that this incurs a multiplicative $\log n$ overhead in communication cost. Further, the protocol in the lemma also gives a $\kappa(G)$-ordering, since it accepts the input $(G, \kappa(G))$. Finally, to obtain a $\kappa(G)$-core, Alice and Bob can run the protocol in the lemma on the input $(G, \kappa(G) - 1)$. □

In the rest of this section, we will prove Lemma 7.2. First, we show an easier (but more expensive) protocol that captures the main idea.

7.1 Warm-Up: An $\widetilde{O}(n\sqrt{n})$ Communication Protocol

Recall the classical peeling algorithm, where one successively peels off the vertex of minimum degree to find a degeneracy ordering of G. We state a decision version of it below, which removes an arbitrary vertex of degree $\leqslant k$ in each step.

Algorithm 1. The Peeling Algorithm.

Input: A graph G, and an integer k.
Output: Accept if G has degeneracy $\leqslant k$, Reject otherwise.

1. While G is non-empty:
   (a) If there is a vertex $v \in G$ of degree $\leqslant k$, remove v from G.
   (b) Else output Reject.
2. Output Accept.

Note that in the Accept case the k-ordering is simply the order the vertices were peeled in, and in the Reject case, the $(k+1)$-core is the set of vertices remaining in G when there are no vertices of degree $\leqslant k$.

We want to somehow port this algorithm to the communication setting. The immediate problem is that Alice or Bob cannot compute a low degree vertex without any communication after a vertex is removed. The naive way to get around this is to send the updated degree of each vertex after each deletion, but this can cost $\Omega(n^2)$ communication. Instead, we observe that we only need to update the degrees of a remaining vertex if it is not too large:

Observation 7.3. *If* $\deg(u) > k + \sqrt{n}$ *at some step of the peeling algorithm, then* u *cannot be deleted in the next* $\sqrt{n}$ *steps, since its degree falls by at most* 1 *per step.*

Hence we have the following algorithm:

1. Compute the set $S = \{u \in V \mid \deg(u) > k + \sqrt{n}\}$ of *safe* vertices, and set it aside.
2. Perform $\sqrt{n}$ iterations of the peeling algorithm, while updating the degree of a neighbor u of a deleted vertex only if $u \notin S$ and $\deg(u) > k$.
3. If the graph is non-empty, go to step 1.

Let $N_A(u)$ and $\deg_A(u)$ denote the neighborhood and degree of u within (V, E_A) (and similarly $N_B(u)$ and $\deg_B(u)$ for E_B). We provide an implementation of the algorithm above in a communication protocol; the sets D and L hold vertices that are ready to *delete*, and *low* degree respectively.

Algorithm 2. The $\widetilde{O}(n\sqrt{n})$ communication protocol for $\mathsf{DEGEN}_{n,k}$.

Input: Alice and Bob get disjoint edge sets E_A and E_B respectively, and both players get an integer k.
Output: Accept if $G = E_A \cup E_B$ has degeneracy $\leqslant k$, Reject otherwise.

1. Alice and Bob send $\deg_A(u)$ and $\deg_B(u)$ respectively for every vertex $u \in V(G)$, so they can then compute:
   - $D \leftarrow \{u \in G \mid \deg(u) \leqslant k\}$.
   - $L \leftarrow \{u \in G \mid k+1 \leqslant \deg(u) \leqslant k + \sqrt{n}\}$.
   - $S \leftarrow V(G) \setminus (D \cup L)$.
2. For $\sqrt{n}$ iterations:
   (a) If D is empty, output Reject.
   (b) Let v be any vertex in D. To remove v from G:
   (c) Alice and Bob send $N_A(v) \cap L$ and $N_B(v) \cap L$ respectively.
   (d) Hence they can update $\deg(u)$ for all $u \in L$.
   (e) If $\deg(u) < k+1$ for any $u \in L$, Alice and Bob move it to D.
3. If G is non-empty, go to Step 1.
4. Output Accept.

Note that the proofs (a k-ordering of the vertices in the Accept case, and a $(k+1)$-core in the Reject case) fall out of the protocol exactly as before. The correctness of this protocol follows from Observation 7.3; we omit the proof here since it is subsumed by the next section. To see that it only uses $\widetilde{O}(n\sqrt{n})$ bits of communication, we observe that:

- Sending the degree of all vertices costs $O(n \log n)$ bits, and we do this at most $\sqrt{n}$ times.
- Once a vertex is in D, no communication is spent for updating its own degree.

- Each vertex in L features at most $\sqrt{n}$ times in the neighborhood of a deleted vertex before it is moved to D, costing $O(\sqrt{n}\log n)$ bits per vertex.

Hence the total communication cost of the protocol is $O(n\sqrt{n}\log n)$.

7.2 An $\widetilde{O}(n)$ Communication Protocol

There are two changes from the previous protocol:

1. Instead of a single safe set of vertices, we have $O(\log n)$ safe sets corresponding to exponentially increasing gaps between the degree and k. More precisely, we define the set S_i for $i \in [\log n]$ such that whenever Alice and Bob compute $\deg(u)$ for some vertex u, they place it in S_i if $k + 2^{i-1} \leqslant \deg(u) < k + 2^i$.

2. Instead of counting a global number of iterations until the safe vertices are updated, we track each vertex individually. Concretely, suppose $\deg(u) \geqslant k + \ell$. Then u cannot be deleted until $\deg_A(u)$ or $\deg_B(u)$ falls by at least $\ell/2$.

The upshot of the two changes above is that for $u \in S_i$ Alice and Bob do not have to update $\deg(u)$ until $\deg_A(u)$ or $\deg_B(u)$ falls by at least 2^{i-2}. We can now state the algorithm. As before, the set D holds all vertices which are ready for deletion.

Algorithm 3. The $\widetilde{O}(n)$ communication protocol for $\mathsf{DEGEN}_{n,k}$.

Input: Alice and Bob get disjoint edge sets E_A and E_B respectively, and both players get an integer k.
Output: Accept if $G = E_A \cup E_B$ has degeneracy $\leqslant k$, Reject otherwise.

1. Alice and Bob send $\deg_A(u)$ and $\deg_B(u)$ respectively for every vertex $u \in V(G)$, so they can then compute:
   - $D \leftarrow \{u \in G \mid \deg(u) \leqslant k\}$.
   - $S_i \leftarrow \left\{u \in G \;\middle|\; k + 2^{i-1} \leqslant \deg(u) < k + 2^i\right\}$, for each $i \in [\log n]$.
2. While G is non-empty:
   (a) If D is empty, output Reject.
   (b) Let v be any vertex in D. To remove v from G:
   (c) For each $i \in [\log n]$, and each $u \in S_i$, if $\deg_A(u)$ or $\deg_B(u)$ has fallen by $\geqslant 2^{i-2}$ since it was last communicated, Alice sends $\deg_A(u)$ and Bob sends $\deg_B(u)$, hence allowing them to update $\deg(u)$.
   (d) For all vertices updated in the previous step, Alice and Bob place them in the correct S_i (or D).
3. Output Accept.

To wrap up, we prove that the algorithm above is correct, and uses $\widetilde{O}(n)$ space.

Proof of Lemma 7.2. To prove that this protocol is correct, we argue that:

- All vertices have degree $\leqslant k$ when they are removed, which implies that if the protocol outputs Accept, $\kappa(G) \leqslant k$. This is immediate because we only move vertices to D when we know their current degree exactly, and it is $\leqslant k$.
- Any vertex u with degree $\leqslant k$ is placed in D, which implies that if $\kappa(G) \leqslant k$, then the protocol outputs Accept. Suppose towards a contradiction that u was placed in the set S_i when its degree was updated for the final time. This means that $\deg(u)$ was at least $k + 2^{i-1}$. But now for $\deg(u)$ to fall below $k + 1$, at least one of $\deg_A(u)$ or $\deg_B(u)$ must fall by $\geqslant 2^{i-2}$ (otherwise $\deg(u)$ falls by less than 2^{i-1}). Hence $\deg(u)$ was updated again following the final update, and we have a contradiction.

Next, we argue that this protocol uses only $\widetilde{O}(n)$ communication.

- As before, the initial setup uses only $O(\log n)$ bits per vertex.
- While in S_i, a vertex can have its degree updated at most twice before it falls below $k + 2^{i-1}$, and is hence dropped from S_i. Since there are only $\log n$ sets S_i, and a vertex can never gain degree, we only update its degree $O(\log n)$ times before it falls into D, spending $O(\log^2 n)$ bits overall.

Hence, in total, the protocol uses only $O(n \log^2 n)$ bits of communication. □

References

[A17] Sepehr Assadi A. Tight space-approximation tradeoff for the multi-pass streaming set cover problem. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS 2017, Chicago, IL, USA, May 14-19, 2017*, pages 321–335, 2017.

[A22] Sepehr Assadi A. A two-pass (conditional) lower bound for semi-streaming maximum matching. In Joseph (Seffi) Naor and Niv Buchbinder, editors, *Proceedings of the 2022 ACM-SIAM Symposium on Discrete Algorithms, SODA 2022, Virtual Conference / Alexandria, VA, USA, January 9 - 12, 2022*, pages 708–742. SIAM, 2022.

[AA20] Noga Alon and Sepehr Assadi. Palette sparsification beyond (Δ+1) vertex coloring. In Jaroslaw Byrka and Raghu Meka, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM 2020, August 17-19, 2020, Virtual Conference*, volume 176 of *LIPIcs*, pages 6:1–6:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. 1

[ACK19] Sepehr Assadi, Yu Chen, and Sanjeev Khanna. Polynomial pass lower bounds for graph streaming algorithms. In *STOC*, pages 265–276. ACM, 2019.

[ACKP21] Amir Abboud, Keren Censor-Hillel, Seri Khoury, and Ami Paz. Smaller cuts, higher lower bounds. *ACM Trans. Algorithms*, 17(4):30:1–30:40, 2021.

[AD21] Sepehr Assadi and Aditi Dudeja. A simple semi-streaming algorithm for global minimum cuts. In *SOSA*, pages 172–180. SIAM, 2021.

[ADBV05] J. Ignacio Alvarez-Hamelin, Luca Dall'Asta, Alain Barrat, and Alessandro Vespignani. Large scale networks fingerprinting and visualization using the k-core decomposition. In *Advances in Neural Information Processing Systems 18 [Neural Information Processing Systems, NIPS 2005, December 5-8, 2005, Vancouver, British Columbia, Canada]*, pages 41–50, 2005. 1

[AJJ+22] Sepehr Assadi, Arun Jambulapati, Yujia Jin, Aaron Sidford, and Kevin Tian. Semi-streaming bipartite matching in fewer passes and optimal space. In *SODA*, pages 627–669. SIAM, 2022.

[AKSY20] Sepehr Assadi, Gillat Kol, Raghuvansh Saxena, and Huacheng Yu. Multi-pass graph streaming lower bounds for cycle counting, max-cut, matching size, and other problems. In *61st Annual IEEE Symposium on Foundations of Computer Science, FOCS (to appear)*, 2020.

[AKZ22] Sepehr Assadi, Gillat Kol, and Zhijun Zhang. Rounds vs communication tradeoffs for maximal independent sets. In *63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022*, pages 1193–1204. IEEE, 2022.

[AN21] Sepehr Assadi and Vishvajeet N. Graph streaming lower bounds for parameter estimation and property testing via a streaming XOR lemma. In Samir Khuller and Virginia Vassilevska Williams, editors, *STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021*, pages 612–625. ACM, 2021.

[AR20] Sepehr Assadi and Ran Raz. Near-quadratic lower bounds for two-pass graph streaming algorithms. In *FOCS*, pages 342–353. IEEE, 2020.

[AYZ97] Noga Alon, Raphael Yuster, and Uri Zwick. Finding and counting given length cycles. *Algorithmica*, 17(3):209–223, 1997. 1

[BBCR10] Boaz Barak, Mark Braverman, Xi Chen, and Anup Rao. How to compress interactive communication. In *Proceedings of the 42nd ACM Symposium on Theory of Computing, STOC 2010, 5-8 June 2010*, pages 67–76, 2010.

[BCD+19] Nir Bachrach, Keren Censor-Hillel, Michal Dory, Yuval Efron, Dean Leitersdorf, and Ami Paz. Hardness of distributed optimization. In Peter Robinson and Faith Ellen, editors, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, pages 238–247. ACM, 2019.

[BCG20] Suman K. Bera, Amit Chakrabarti, and Prantar Ghosh. Graph coloring via degeneracy in streaming and other space-conscious models. In *47th International Colloquium on Automata, Languages, and Programming, ICALP 2020, July 8-11, 2020, Saarbrücken, Germany (Virtual Conference)*, pages 11:1–11:21, 2020. 1, 2

[BEO+13] Mark Braverman, Faith Ellen, Rotem Oshman, Toniann Pitassi, and Vinod Vaikuntanathan. A tight bound for set disjointness in the message-passing model. In *54th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2013, 26-29 October, 2013, Berkeley, CA, USA*, pages 668–677, 2013.

[BGKV14] Francesco Bonchi, Francesco Gullo, Andreas Kaltenbrunner, and Yana Volkovich. Core decomposition of uncertain graphs. In Sofus A. Macskassy, Claudia Perlich, Jure Leskovec, Wei Wang, and Rayid Ghani, editors, *The 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '14, New York, NY, USA - August 24 - 27, 2014*, pages 1316–1325. ACM, 2014. 1

[BHNT15] Sayan Bhattacharya, Monika Henzinger, Danupon Nanongkai, and Charalampos E. Tsourakakis. Space- and time-efficient algorithm for maintaining dense subgraphs on one-pass dynamic streams. In *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing, STOC 2015, Portland, OR, USA, June 14-17, 2015*, pages 173–182, 2015. 1

[BJKS02] Ziv Bar-Yossef, T. S. Jayram, Ravi Kumar, and D. Sivakumar. An information statistics approach to data stream and communication complexity. In *43rd Symposium on Foundations of Computer Science (FOCS 2002), 16-19 November 2002, Proceedings*, pages 209–218, 2002.

[BKS02] Ziv Bar-Yossef, Ravi Kumar, and D. Sivakumar. Reductions in streaming algorithms, with an application to counting triangles in graphs. In *Proceedings of the Thirteenth Annual ACM-SIAM Symposium on Discrete Algorithms, January 6-8, 2002, San Francisco, CA, USA.*, pages 623–632, 2002.

[BKV12] Bahman Bahmani, Ravi Kumar, and Sergei Vassilvitskii. Densest subgraph in streaming and mapreduce. *PVLDB*, 5(5):454–465, 2012. 1

[BR11] Mark Braverman and Anup Rao. Information equals amortized communication. In *IEEE 52nd Annual Symposium on Foundations of Computer Science, FOCS 2011, October 22-25, 2011*, pages 748–757, 2011.

[BvdBE+22] Joakim Blikstad, Jan van den Brand, Yuval Efron, Sagnik Mukhopadhyay, and Danupon Nanongkai. Nearly optimal communication and query complexity of bipartite matching. In *FOCS*, pages 1174–1185. IEEE, 2022.

[CFHT20] Yi-Jun Chang, Martin Farach-Colton, Tsan-sheng Hsu, and Meng-Tsung Tsai. Streaming complexity of spanning tree computation. In *37th International Symposium on Theoretical Aspects of Computer Science, STACS 2020, March 10-13, 2020, Montpellier, France*, pages 34:1–34:19, 2020.

[CGMV20] Amit Chakrabarti, Prantar Ghosh, Andrew McGregor, and Sofya Vorotnikova. Vertex ordering problems in directed graph streams. In *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms, SODA 2020, Salt Lake City, UT, USA, January 5-8, 2020*, pages 1786–1802, 2020.

[Cha00] Moses Charikar. Greedy approximation algorithms for finding dense components in a graph. In Klaus Jansen and Samir Khuller, editors, *Approximation Algorithms for Combinatorial Optimization, Third International Workshop, APPROX 2000, Saarbrücken, Germany, September 5-8, 2000, Proceedings*, volume 1913 of *Lecture Notes in Computer Science*, pages 84–95. Springer, 2000. 1

[CKP+21a] Lijie Chen, Gillat Kol, Dmitry Paramonov, Raghuvansh R. Saxena, Zhao Song, and Huacheng Yu. Almost optimal super-constant-pass streaming lower bounds for reachability. In Samir Khuller and Virginia Vassilevska Williams, editors, *STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021*, pages 570–583. ACM, 2021.

[CKP+21b] Lijie Chen, Gillat Kol, Dmitry Paramonov, Raghuvansh R. Saxena, Zhao Song, and Huacheng Yu. Near-optimal two-pass streaming algorithm for sampling random walks over directed graphs. In Nikhil Bansal, Emanuela Merelli, and James Worrell, editors,

48th International Colloquium on Automata, Languages, and Programming, ICALP 2021, July 12-16, 2021, Glasgow, Scotland (Virtual Conference), volume 198 of *LIPIcs*, pages 52:1–52:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

[CKP+23] Lijie Chen, Gillat Kol, Dmitry Paramonov, Raghuvansh R. Saxena, Zhao Song, and Huacheng Yu. Towards multi-pass streaming lower bounds for optimal approximation of max-cut. In Nikhil Bansal and Viswanath Nagarajan, editors, *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*, pages 878–924. SIAM, 2023.

[CSWY01] Amit Chakrabarti, Yaoyun Shi, Anthony Wirth, and Andrew Chi-Chih Yao. Informational complexity and the direct sum problem for simultaneous message complexity. In *42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14-17 October 2001*, pages 270–278, 2001.

[CT06] Thomas M. Cover and Joy A. Thomas. *Elements of information theory (2. ed.)*. Wiley, 2006.

[CZL+20] Deming Chu, Fan Zhang, Xuemin Lin, Wenjie Zhang, Ying Zhang, Yinglong Xia, and Chenyi Zhang. Finding the best k in core decomposition: A time and space optimal solution. In *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020*, pages 685–696. IEEE, 2020. 1

[DBS17] Laxman Dhulipala, Guy E. Blelloch, and Julian Shun. Julienne: A framework for parallel graph algorithms using work-efficient bucketing. In Christian Scheideler and Mohammad Taghi Hajiaghayi, editors, *Proceedings of the 29th ACM Symposium on Parallelism in Algorithms and Architectures, SPAA 2017, Washington DC, USA, July 24-26, 2017*, pages 293–304. ACM, 2017. 1

[DBS18] Laxman Dhulipala, Guy E. Blelloch, and Julian Shun. Theoretically efficient parallel graph algorithms can be fast and scalable. In Christian Scheideler and Jeremy T. Fineman, editors, *Proceedings of the 30th on Symposium on Parallelism in Algorithms and Architectures, SPAA 2018, Vienna, Austria, July 16-18, 2018*, pages 393–404. ACM, 2018. 1

[DNO19] Shahar Dobzinski, Noam Nisan, and Sigal Oren. Economic efficiency requires interaction. *Games Econ. Behav.*, 118:589–608, 2019.

[EH66] Paul Erdős and András Hajnal. On chromatic number of graphs and set-systems. *Acta Math. Acad. Sci. Hungar*, 17(61-99):1, 1966. 1

[ELM18] Hossein Esfandiari, Silvio Lattanzi, and Vahab S. Mirrokni. Parallel and streaming algorithms for k-core decomposition. In Jennifer G. Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pages 1396–1405. PMLR, 2018. 1

[FHW12] Silvio Frischknecht, Stephan Holzer, and Roger Wattenhofer. Networks cannot compute their diameter in sublinear time. In Yuval Rabani, editor, *Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2012, Kyoto, Japan, January 17-19, 2012*, pages 1150–1162. SIAM, 2012.

[FKM+05] Joan Feigenbaum, Sampath Kannan, Andrew McGregor, Siddharth Suri, and Jian Zhang. On graph problems in a semi-streaming model. *Theor. Comput. Sci.*, 348(2-3):207–216, 2005.

[FT14] Martin Farach-Colton and Meng-Tsung Tsai. Computing the degeneracy of large graphs. In Alberto Pardo and Alfredo Viola, editors, *LATIN 2014: Theoretical Informatics - 11th Latin American Symposium, Montevideo, Uruguay, March 31 - April 4, 2014. Proceedings*, volume 8392 of *Lecture Notes in Computer Science*, pages 250–260. Springer, 2014. 1

[FT16] Martin Farach-Colton and Meng-Tsung Tsai. Tight approximations of degeneracy in large graphs. In *LATIN 2016: Theoretical Informatics - 12th Latin American Symposium, Ensenada, Mexico, April 11-15, 2016, Proceedings*, pages 429–440, 2016. 1

[GBGL20] Edoardo Galimberti, Francesco Bonchi, Francesco Gullo, and Tommaso Lanciano. Core decomposition in multilayer networks: Theory, algorithms, and applications. *ACM Trans. Knowl. Discov. Data*, 14(1):11:1–11:40, 2020. 1

[GLM19] Mohsen Ghaffari, Silvio Lattanzi, and Slobodan Mitrovic. Improved parallel algorithms for density-based network clustering. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pages 2201–2210. PMLR, 2019. 1

[GO13] Venkatesan Guruswami and Krzysztof Onak. Superlinear lower bounds for multipass graph processing. In *Proceedings of the 28th Conference on Computational Complexity, CCC 2013, K.lo Alto, California, USA, 5-7 June, 2013*, pages 287–298, 2013.

[HSSW12] Magnús M. Halldórsson, Xiaoming Sun, Mario Szegedy, and Chengu Wang. Streaming and communication complexity of clique approximation. In *Automata, Languages, and Programming - 39th International Colloquium, ICALP 2012, Warwick, UK, July 9-13, 2012, Proceedings, Part I*, pages 449–460, 2012.

[IKL+12] Gábor Ivanyos, Hartmut Klauck, Troy Lee, Miklos Santha, and Ronald de Wolf. New bounds on the classical and quantum communication complexity of some graph properties. In *FSTTCS*, volume 18 of *LIPIcs*, pages 148–159. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2012.

[JKS03] T. S. Jayram, Ravi Kumar, and D. Sivakumar. Two applications of information complexity. In *STOC*, pages 673–682. ACM, 2003.

[KBVT15] Wissam Khaouid, Marina Barsky, S. Venkatesh, and Alex Thomo. K-core decomposition of large networks on a single PC. *Proc. VLDB Endow.*, 9(1):13–23, 2015. 1

[Kla00] Hartmut Klauck. On quantum and probabilistic communication: Las vegas and one-way protocols. In *32nd Annual ACM Symposium on Theory of Computing (STOC)*, pages 644–651, 2000.

[KN97] Eyal Kushilevitz and Noam Nisan. *Communication complexity*. Cambridge University Press, 1997.

[KPP16] Tsvi Kopelowitz, Seth Pettie, and Ely Porat. Higher lower bounds from the 3sum conjecture. In Robert Krauthgamer, editor, *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2016, Arlington, VA, USA, January 10-12, 2016*, pages 1272–1287. SIAM, 2016.

[KPSY23] Gillat Kol, Dmitry Paramonov, Raghuvansh R. Saxena, and Huacheng Yu. Characterizing the multi-pass streaming complexity for solving boolean csps exactly. In Yael Tauman Kalai, editor, *14th Innovations in Theoretical Computer Science Conference, ITCS 2023, January 10-13, 2023, MIT, Cambridge, Massachusetts, USA*, volume 251 of *LIPIcs*, pages 80:1–80:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.

[LJS19] Yang P. Liu, Arun Jambulapati, and Aaron Sidford. Parallel reachability in almost linear work and square root depth. In David Zuckerman, editor, *60th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2019, Baltimore, Maryland, USA, November 9-12, 2019*, pages 1664–1686. IEEE Computer Society, 2019.

[LSY+22] Quanquan C. Liu, Jessica Shi, Shangdi Yu, Laxman Dhulipala, and Julian Shun. Parallel batch-dynamic algorithms for k-core decomposition and related graph problems. In Kunal Agrawal and I-Ting Angelina Lee, editors, *SPAA '22: 34th ACM Symposium on Parallelism in Algorithms and Architectures, Philadelphia, PA, USA, July 11 - 14, 2022*, pages 191–204. ACM, 2022. 1

[LWSX18] Chao Li, Li Wang, Shiwen Sun, and Chengyi Xia. Identification of influential spreaders based on classified neighbors in real-world complex networks. *Appl. Math. Comput.*, 320:512–523, 2018. 1

[LZZ+19] Conggai Li, Fan Zhang, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. Efficient progressive minimum k-core search. *Proc. VLDB Endow.*, 13(3):362–375, 2019. 1

[MB83] David W. Matula and Leland L. Beck. Smallest-last ordering and clustering and graph coloring algorithms. *J. ACM*, 30(3):417–427, 1983. 1, 2

[McG14] Andrew McGregor. Graph stream algorithms: a survey. *SIGMOD Rec.*, 43(1):9–20, 2014.

[MN20] Sagnik Mukhopadhyay and Danupon Nanongkai. Weighted min-cut: sequential, cut-query, and streaming algorithms. In *STOC*, pages 496–509. ACM, 2020.

[MTVV15] Andrew McGregor, David Tench, Sofya Vorotnikova, and Hoa T. Vu. Densest subgraph in dynamic graph streams. In *Mathematical Foundations of Computer Science 2015 - 40th International Symposium, MFCS 2015, Milan, Italy, August 24-28, 2015, Proceedings, Part II*, pages 472–482, 2015. 1

[NW91] Noam Nisan and Avi Wigderson. Rounds in communication complexity revisited. In *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing, May 5-8, 1991, New Orleans, Louisiana, USA*, pages 419–429, 1991.

[NW93] Noam Nisan and Avi Wigderson. Rounds in communication complexity revisited. *SIAM J. Comput.*, 22(1):211–219, 1993.

[PRV99] Stephen Ponzio, Jaikumar Radhakrishnan, and Srinivasan Venkatesh. The communication complexity of pointer chasing: Applications of entropy and sampling. In

Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing, May 1-4, 1999, Atlanta, Georgia, USA, pages 602–611, 1999.

[PS84] Christos H. Papadimitriou and Michael Sipser. Communication complexity. *J. Comput. Syst. Sci.*, 28(2):260–269, 1984.

[Raz92] Alexander A. Razborov. On the distributional complexity of disjointness. *Theor. Comput. Sci.*, 106(2):385–390, 1992.

[RSW18] Aviad Rubinstein, Tselil Schramm, and S. Matthew Weinberg. Computing exact minimum cuts without knowing the graph. In *ITCS*, volume 94 of *LIPIcs*, pages 39:1–39:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.

[RW13] Liam Roditty and Virginia Vassilevska Williams. Fast approximation algorithms for the diameter and radius of sparse graphs. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *Symposium on Theory of Computing Conference, STOC'13, Palo Alto, CA, USA, June 1-4, 2013*, pages 515–524. ACM, 2013.

[SGJ+13] Ahmet Erdem Sariyüce, Bugra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Ümit V. Çatalyürek. Streaming algorithms for k-core decomposition. *Proc. VLDB Endow.*, 6(6):433–444, 2013. 1

[Yao79] Andrew Chi-Chih Yao. Some complexity questions related to distributive computing (preliminary report). In *Proceedings of the 11h Annual ACM Symposium on Theory of Computing, April 30 - May 2, 1979, Atlanta, Georgia, USA*, pages 209–213, 1979.

[Yeh20] Amir Yehudayoff. Pointer chasing via triangular discrimination. *Comb. Probab. Comput.*, 29(4):485–494, 2020.